AUTOMOTIVE PARTS STORE

A Full-Stack E-Commerce Web Application for Automobile Spare Parts

Prepared by:

[POLAGONI TEJA]

Executive Summary

This report provides details of the Industrial Internship provided by Upskill Campus and UniConverge Technologies Pvt. Ltd. (UCT). The internship focused on a project/problem statement provided by UCT, to be completed along with this report within 6 weeks.

My project was the Automotive Parts Store — a secure, full-stack e-commerce web application that allows customers to browse, search, and purchase automobile spare parts online, while giving administrators complete control over products, orders, customers, and store settings. It was built using Node.js, Express.js, MySQL, HTML, CSS, JavaScript, and Bootstrap, following the MVC (Model-View-Controller) architecture.

This internship gave me a very good opportunity to get exposure to industrial problems and design/implement a solution for that. It was an overall great experience to have this internship.

Table of Contents

1. Preface 4
2. Introduction 4
    2.1 About UniConverge Technologies Pvt. Ltd. 4
    2.2 About Upskill Campus 5
    2.3 Objective 5
    2.4 Reference 5
    2.5 Glossary 6
3. Problem Statement 7
4. Existing and Proposed Solution 7
    4.1 Existing System 7
    4.2 Proposed System 7
5. Technologies Used 8
    5.1 Frontend 8
    5.2 Backend 8
    5.3 Database 8
    5.4 Packages Used 8
    5.5 Tools Used 9
6. Proposed Design / Model 10
    6.1 High Level Diagram 10
    6.2 MVC Architecture Diagram 10
    6.3 Low Level Diagram 11
    6.4 ER Diagram (Database Design) 13
    6.5 Interfaces 16
7. Application Screenshots — Customer Module 17
    7.1 User Registration 17
    7.2 User Login & Logout 17
    7.3 Home Page 18
    7.4 Product Browsing & Search 18
    7.5 Shopping Cart 19
    7.6 Checkout — Delivery Address 19
    7.7 Checkout — Payment 20
    7.8 Order Confirmation 20

7.9 My Orders 21
7.10 Order Tracking 21
8. Application Screenshots — Admin Module 23
8.1 Dashboard 23
8.2 Product Management 23
8.3 Order Management 24
8.4 Customer Management 24
8.5 Reports 25
8.6 Notifications 25
8.7 Store Settings 26
9. Application Workflow 27
9.1 Customer Order Workflow 27
9.2 Admin Order Fulfillment Workflow 27
10. Security Features 27
11. Folder Structure & GitHub Repository 28
12. Performance Test 29
12.1 Test Plan / Test Cases 29
12.2 Test Procedure 29
12.3 Performance Outcome 29
13. My Learnings 30
14. Future Work Scope 30
15. Conclusion 31

1. Preface

This report presents the work carried out during my Industrial Internship under Upskill Campus in collaboration with UniConverge Technologies Pvt. Ltd. (UCT). The internship provided me with an opportunity to gain practical knowledge in Full Stack Web Development by developing a real-world web application.

During the internship, I designed and developed an Automotive Parts Store, an online e-commerce platform for purchasing automobile spare parts. The application was built using Node.js, Express.js, MySQL, HTML, CSS, JavaScript, and Bootstrap.

The project includes secure user authentication, product management, shopping cart functionality, wishlist, order management, profile management, and a complete administrator dashboard. Security features such as password hashing using bcrypt, session management, route authorization, Helmet middleware, rate limiting, and environment variables were also implemented.

This internship enhanced my understanding of backend development, database management, REST APIs, GitHub version control, debugging, and software deployment. I am thankful to the mentors at Upskill Campus and UniConverge Technologies for their continuous guidance throughout this project.

2. Introduction

The Automotive Parts Store is a full-stack web application developed to simplify the process of purchasing automobile spare parts online. The application provides a user-friendly interface where customers can browse products, search spare parts, filter products by vehicle type, add products to the shopping cart, place orders, and track deliveries.

The system also includes a dedicated administrator dashboard where the administrator can manage products, customers, categories, orders, reports, notifications, and website settings.

The project follows the MVC (Model-View-Controller) architecture and uses MySQL for data storage. The objective is to provide a secure, efficient, and scalable online automobile spare parts management system.

2.1 About UniConverge Technologies Pvt. Ltd.

UniConverge Technologies Pvt. Ltd. (UCT) is a technology company focused on providing software development, cloud computing, artificial intelligence, data analytics, cybersecurity, and web application development solutions.

The organization provides internship opportunities to students where they gain practical exposure to industry-standard technologies, software development methodologies, and project implementation.

During the internship, UniConverge Technologies provided technical guidance, project requirements, and mentorship throughout the project development lifecycle.

2.2 About Upskill Campus

Upskill Campus is an online learning platform that offers industrial internships, certification programs, and skill development courses for students and professionals.

It collaborates with reputed industries and organizations to provide hands-on project experience in domains such as:

- Full Stack Web Development
- Artificial Intelligence
- Machine Learning
- Data Science
- Cyber Security
- Cloud Computing

The internship program helped me improve my technical knowledge and practical implementation skills through real-time project development.

2.3 Objective

The main objectives of the project are:

- Develop an online automobile spare parts shopping platform.
- Provide secure customer registration and login.
- Allow users to browse and search products.
- Implement category and vehicle-based product filtering.
- Enable customers to manage shopping carts and wishlists.
- Allow customers to place and track orders.
- Develop a complete administrator dashboard.
- Manage products, customers, and orders efficiently.
- Ensure application security using authentication and authorization.
- Deploy the project using GitHub and cloud hosting.

2.4 Reference

Documentation

- Node.js Official Documentation
- Express.js Documentation
- MySQL Documentation
- Bootstrap Documentation
- MDN Web Docs
- Express Session Documentation
- Helmet Documentation
- Express Rate Limit Documentation
- Multer Documentation
- Bcrypt Documentation
- Git Documentation
- GitHub Documentation

Development Tools

- Visual Studio Code
- MySQL Workbench
- Postman
- Git
- GitHub
- Google Chrome

Project Repository

The complete source code for this project is hosted on GitHub: https://github.com/polagonitejagoud/Automotive-Parts-Store.git

2.5 Glossary

Term	Description
Node.js	JavaScript runtime used for backend development
Express.js	Web framework for Node.js
MySQL	Relational database management system
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JavaScript	Programming language for frontend functionality

Bootstrap	CSS framework for responsive design
MVC	Model View Controller Architecture
REST API	Web services used for communication between frontend and backend
Session	Stores user login information
bcrypt	Library used for password hashing
Helmet	Middleware for securing HTTP headers
Rate Limiter	Protects application from brute-force attacks
Multer	Middleware used for image uploads
CRUD	Create, Read, Update and Delete operations

3. Problem Statement

Traditional automobile spare parts stores require customers to physically visit shops, resulting in time consumption, limited product availability, and difficulty comparing products.

Many small automobile stores also struggle with inventory management, customer management, and order tracking due to manual processes.

The objective of this project is to develop a secure web application that allows customers to purchase automobile spare parts online while providing administrators with a centralized dashboard to manage products, orders, customers, and reports efficiently.

4. Existing and Proposed Solution

4.1 Existing System

The traditional system has several limitations:

- Manual inventory management
- No centralized customer database
- Limited product search capability
- No online ordering
- No order tracking
- Limited product categorization
- Poor inventory monitoring

4.2 Proposed System

The proposed Automotive Parts Store provides:

Customer Features

- Secure Registration
- Login
- Product Browsing
- Product Search
- Vehicle Type Filter
- Shopping Cart
- Wishlist
- Checkout
- Order Tracking

- Profile Management

Administrator Features

- Dashboard
- Product Management
- Category Management
- Customer Management
- Order Management
- Reports
- Notifications
- Store Settings

Advantages

- Secure authentication
- Faster product search
- Efficient inventory management
- Easy order management
- Better customer experience
- Improved administrator control

5. Technologies Used

5.1 Frontend

- HTML5
- CSS3
- Bootstrap
- JavaScript

5.2 Backend

- Node.js
- Express.js

5.3 Database

- MySQL

5.4 Packages Used

- Express

- Express Session
- Express Rate Limit
- Helmet
- Bcrypt
- Multer
- Dotenv
- MySQL2

5.5 Tools Used

- Visual Studio Code
- MySQL Workbench
- Git
- GitHub

6. Proposed Design / Model

The project follows the MVC (Model-View-Controller) architecture, using MySQL for data storage and Express.js for routing and business logic.

6.1 High Level Diagram

The high-level diagram below shows how a request flows from the customer or admin, through the frontend and Express.js server, before reaching the MySQL database.

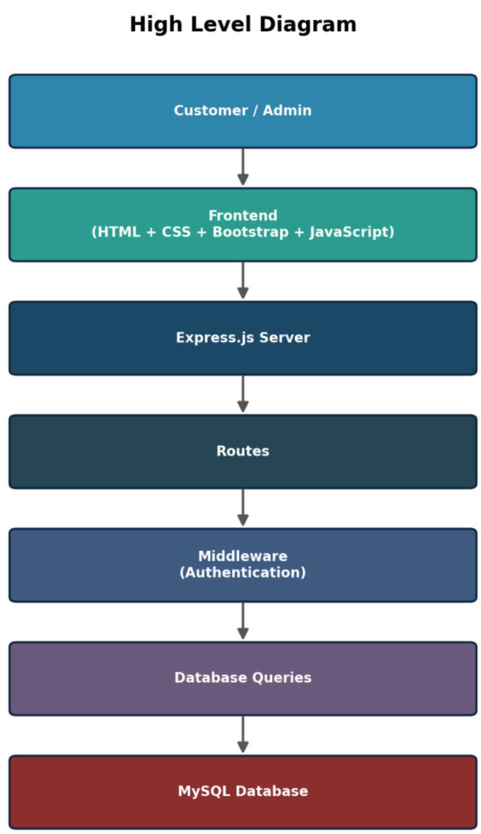


Figure 1: High Level Diagram

6.2 MVC Architecture Diagram

The application is structured using the MVC pattern. The View layer renders HTML/CSS/JS pages, the Controller layer (Express routes and controllers) handles requests and business logic, and the Model layer manages MySQL data access.

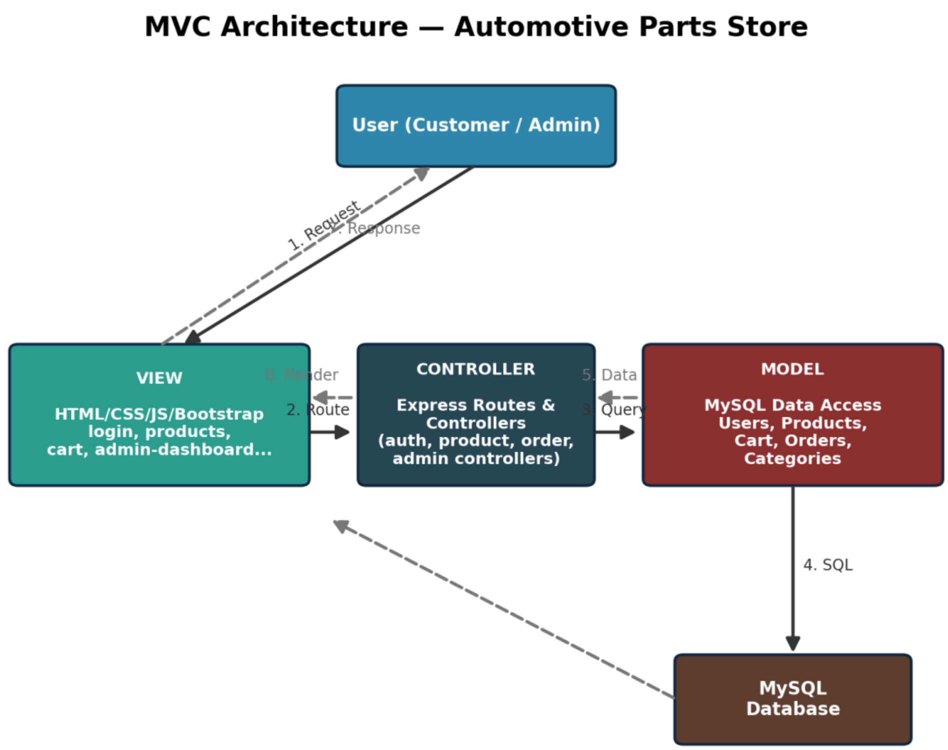


Figure 2: MVC Architecture of the Automotive Parts Store

6.3 Low Level Diagram

The low-level diagram traces the detailed step-by-step flow of the application, from user registration through to admin-side reporting and notifications.

Low Level Diagram

Figure 3: Low Level Diagram

6.4 ER Diagram (Database Design)

The application uses MySQL as the relational database. The Entity Relationship diagram below shows the core tables — Users, Categories, Products, Cart, Orders, Order Items, and Settings — and how they relate to one another.

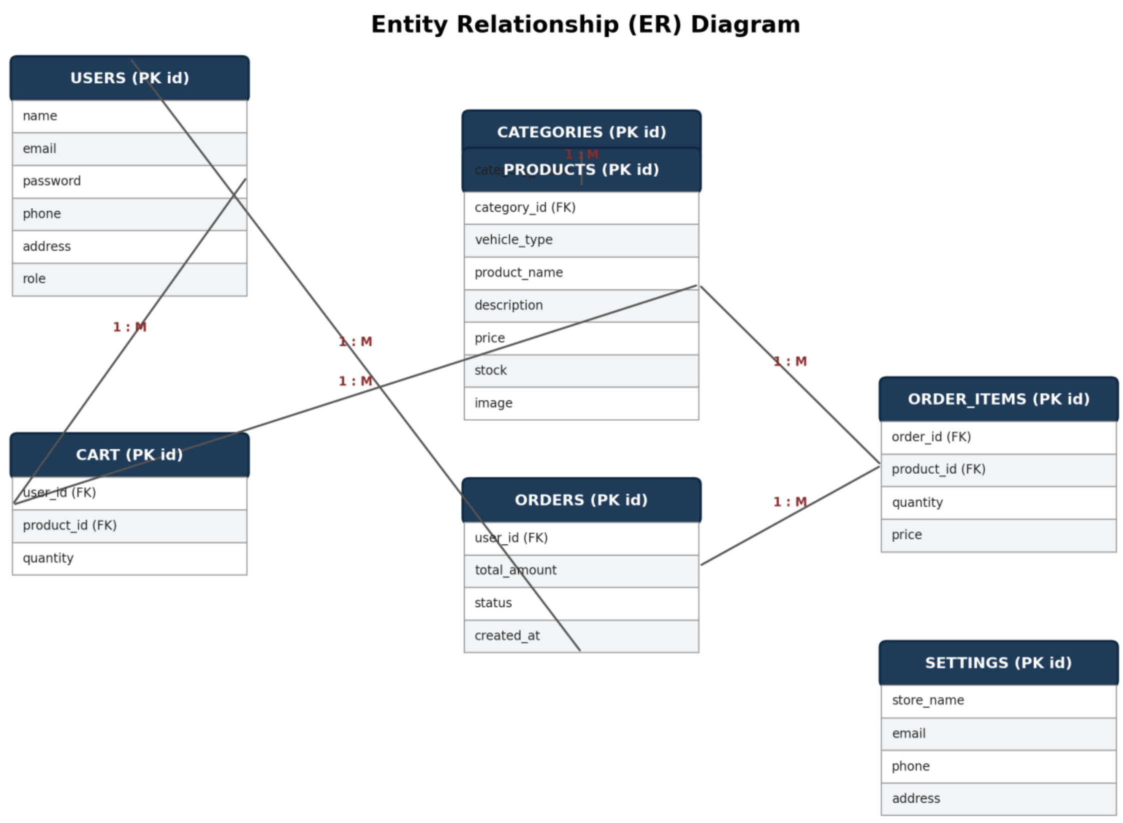


Figure 4: Entity Relationship (ER) Diagram

Users

Stores customer and administrator information.

Fields
id (PK)
name
email
password

phone
address
role

Products

Stores automobile spare parts.

Fields
id (PK)
category_id (FK)
vehicle_type
product_name
description
price
stock
image

Categories

Stores product categories, e.g. Engine Parts, Brake Parts, Electrical Parts, Accessories.

Cart

Stores products added to a customer's cart.

Fields
id (PK)
user_id (FK)
product_id (FK)
quantity

Orders

Stores customer orders.

Fields
id (PK)
user_id (FK)
total_amount
status
created_at

Order Items

Stores individual products within each order.

Fields
id (PK)
order_id (FK)
product_id (FK)
quantity
price

Settings

Stores website information such as Store Name, Email, Phone, and Address.

6.5 Interfaces

Customer Interfaces

- Login Page
- Registration Page
- Home Page
- Products Page
- Product Details
- Shopping Cart
- Wishlist
- Address Page
- Profile Page
- Orders Page
- Payment Page
- Order Tracking
- Order Confirmation

Administrator Interfaces

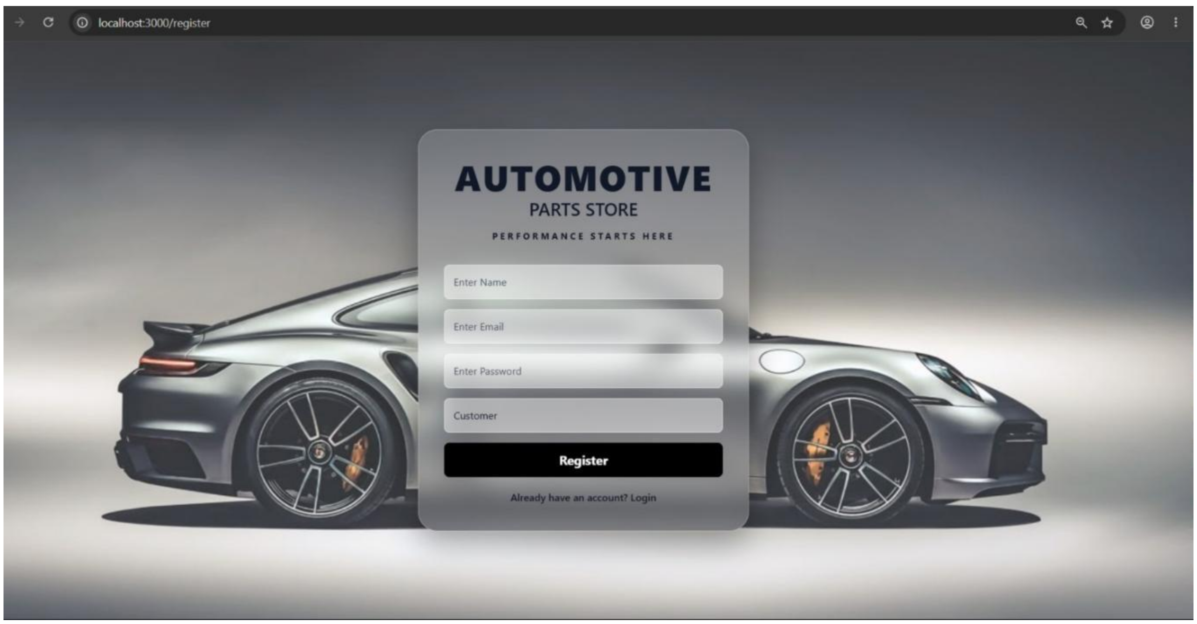
- Dashboard
- Add Product
- Edit Product
- Customers
- Categories
- Orders
- Reports
- Notifications
- Settings

7. Application Screenshots — Customer Module

7.1 User Registration

Users can create an account using Name, Email, and Password. Passwords are encrypted using bcrypt before being stored in MySQL.

Registration Page

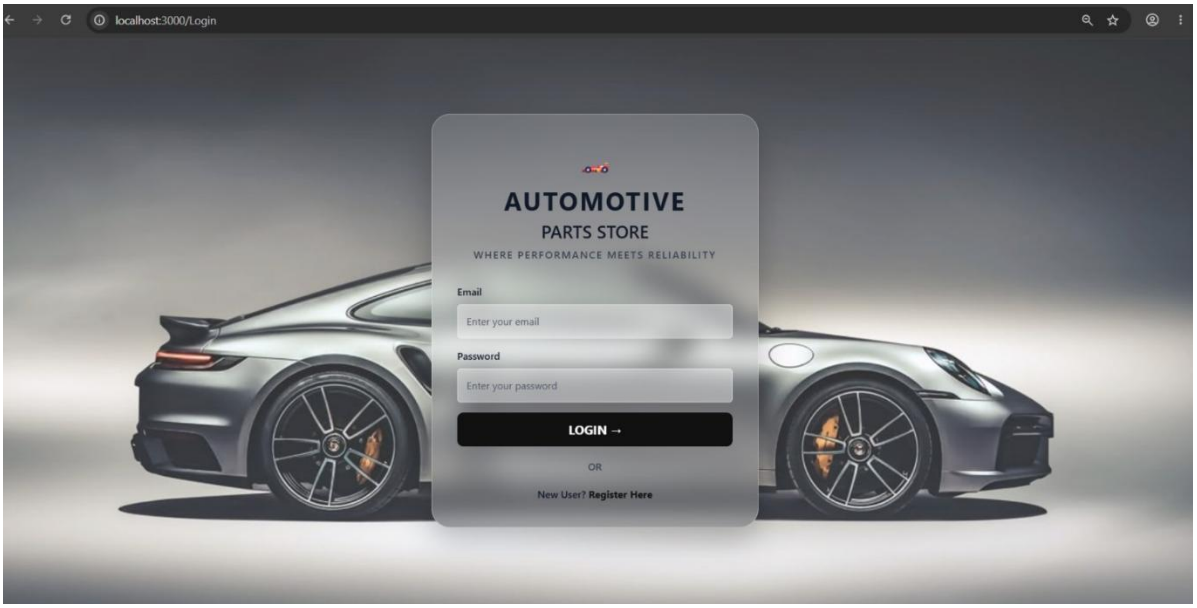


Screenshot: New user registration form

7.2 User Login & Logout

Users can securely log in using their Email and Password. Sessions are maintained using Express Session, and destroyed securely on logout.

Login Page

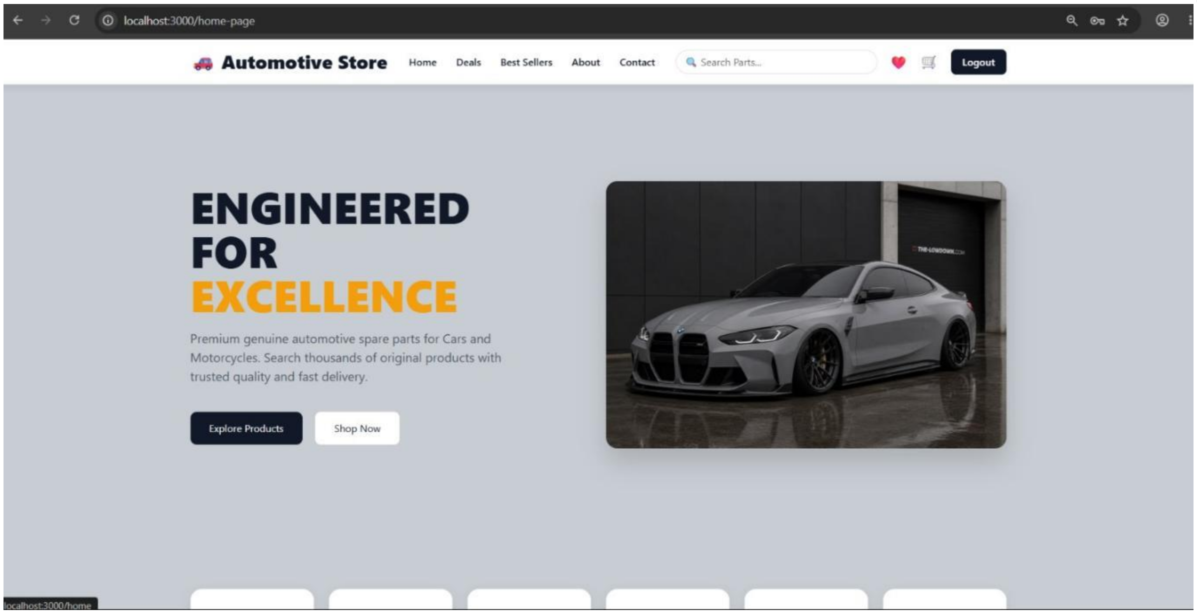


Screenshot: Secure user login form

7.3 Home Page

The landing page showcases featured products and store navigation for customers.

Home Page

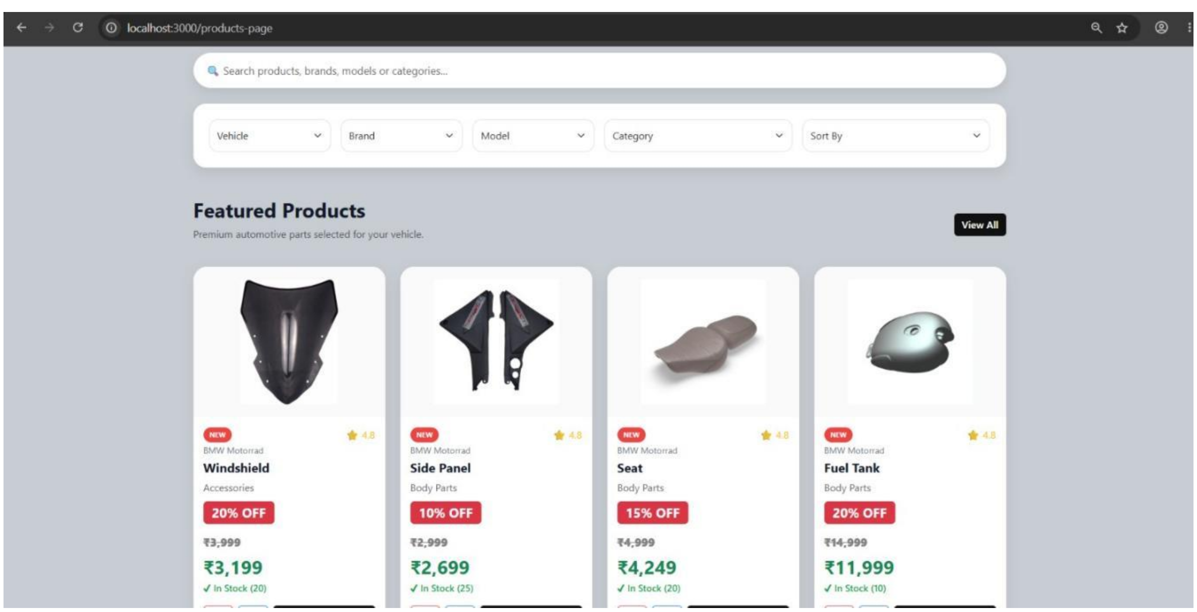


Screenshot: Automotive Store home page

7.4 Product Browsing & Search

Customers can view all products, search products, filter products by vehicle type, and view product details and images.

Products Page

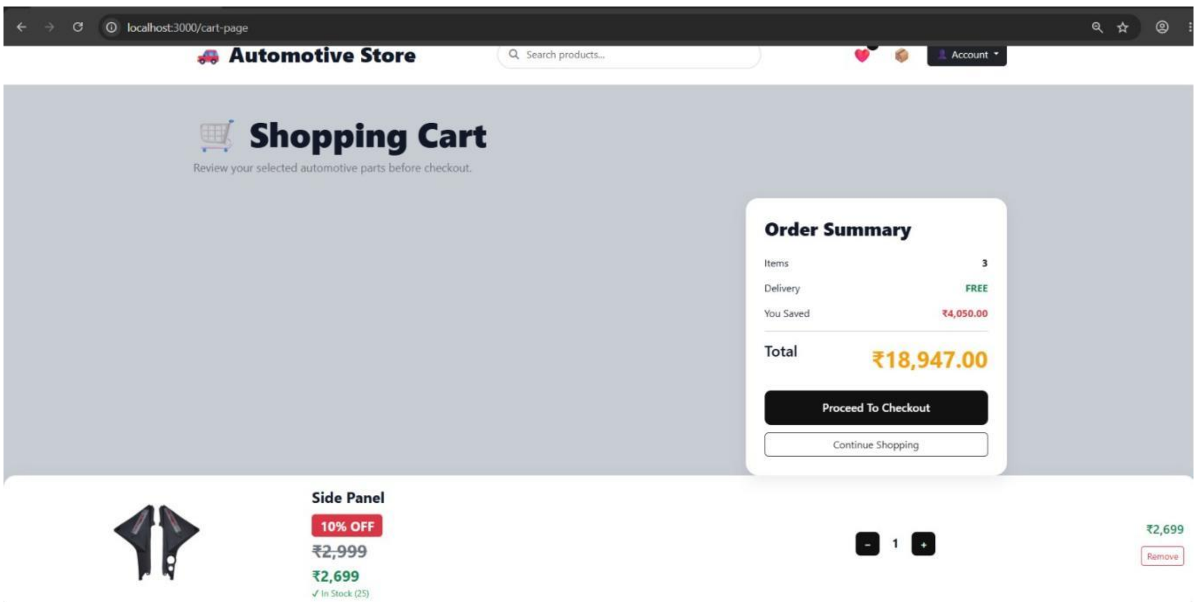


Screenshot: Product listing with search and filters

7.5 Shopping Cart

Customers can add products, remove products, increase or decrease quantity, and view the total amount before checkout.

Shopping Cart

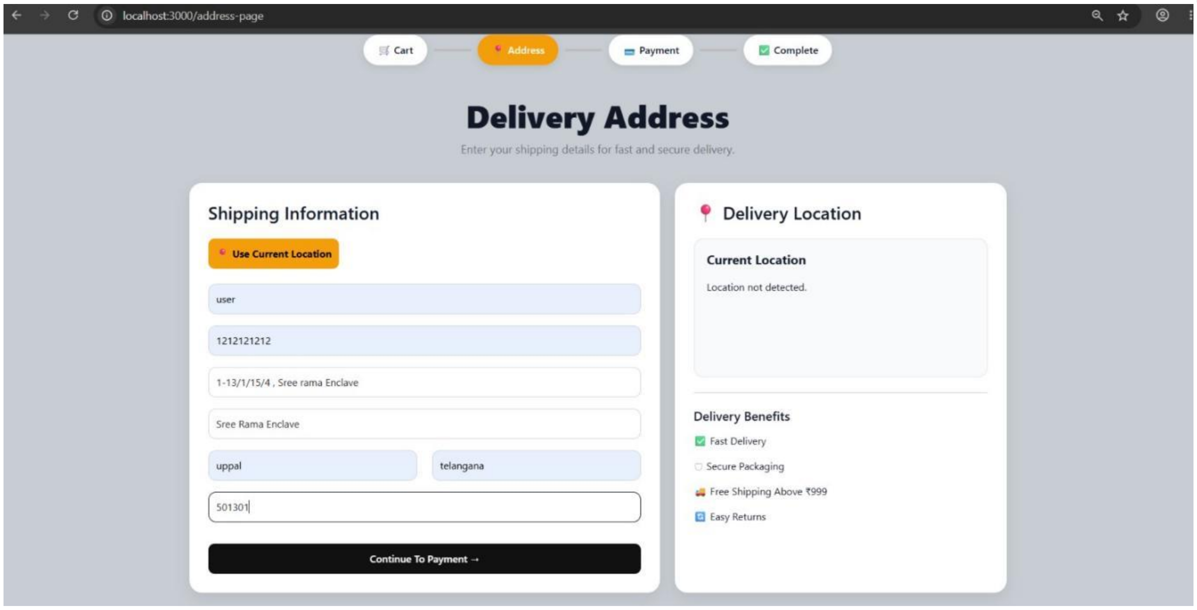


Screenshot: Shopping cart with order summary

7.6 Checkout — Delivery Address

Customers enter their shipping details, with an option to use their current location for faster delivery.

Delivery Address Page

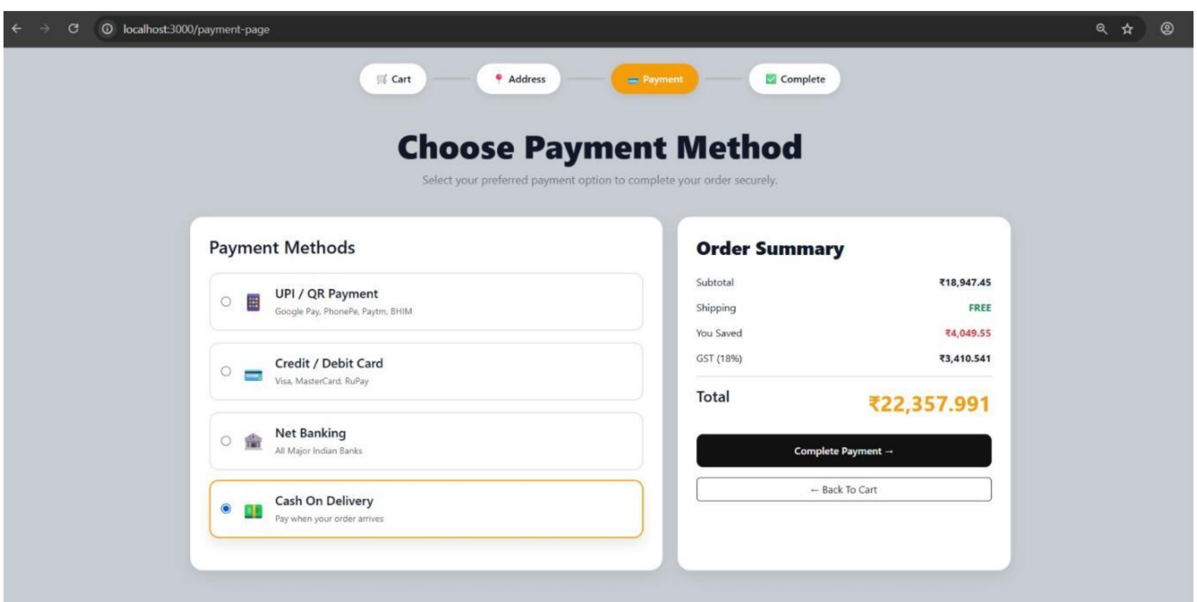


Screenshot: Delivery address entry during checkout

7.7 Checkout — Payment

Customers can choose from multiple payment options: UPI/QR Payment, Credit/Debit Card, Net Banking, or Cash on Delivery.

Payment Page

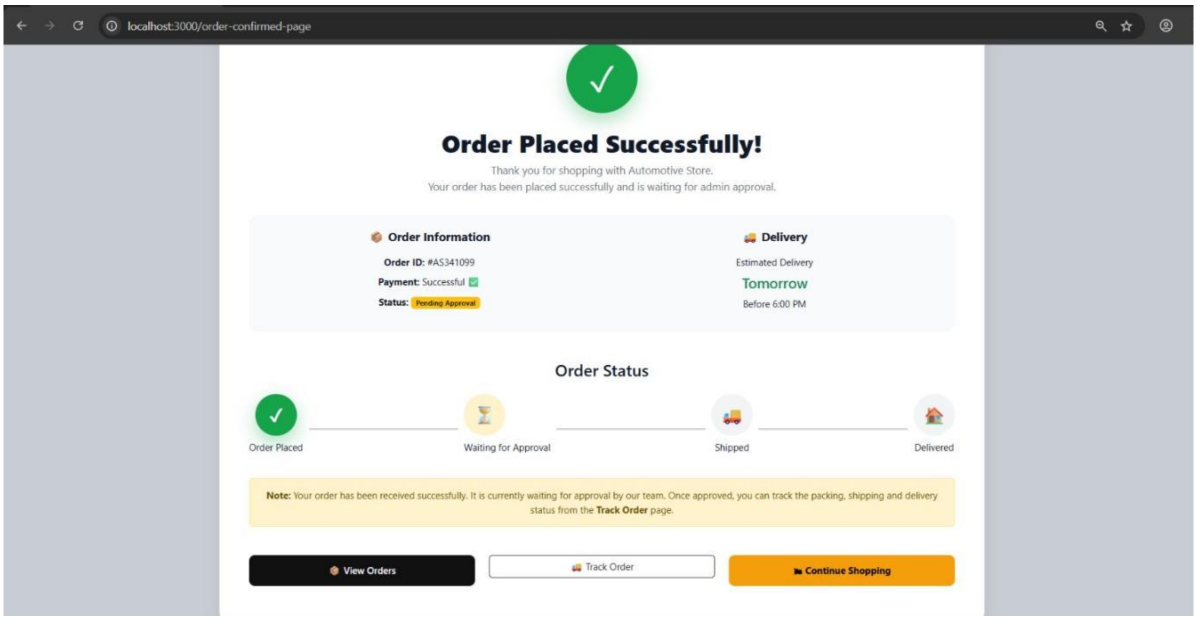


Screenshot: Payment method selection and order summary

7.8 Order Confirmation

Once an order is placed, customers receive an order confirmation with order ID, payment status, and estimated delivery.

Order Confirmed Page

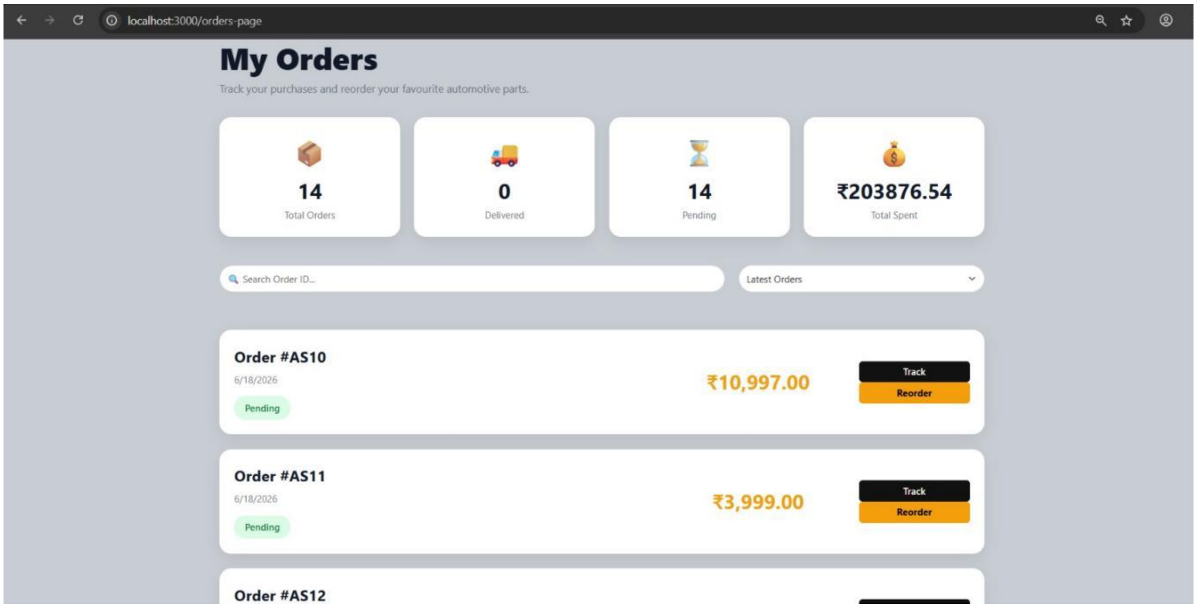


Screenshot: Order placed successfully confirmation

7.9 My Orders

Customers can view their previous orders, track order status, and reorder favourite products.

My Orders Page

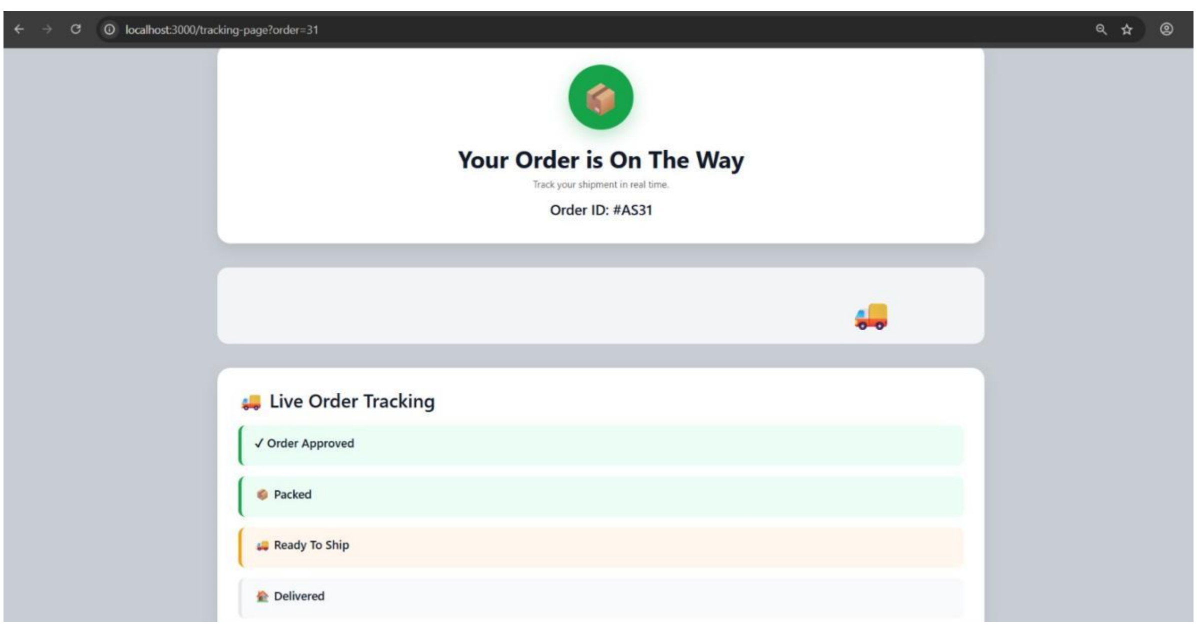


Screenshot: Customer order history

7.10 Order Tracking

Customers can track their order status in real time — from Order Approved to Packed, Ready to Ship, and Delivered.

Live Order Tracking



Screenshot: Live order tracking page

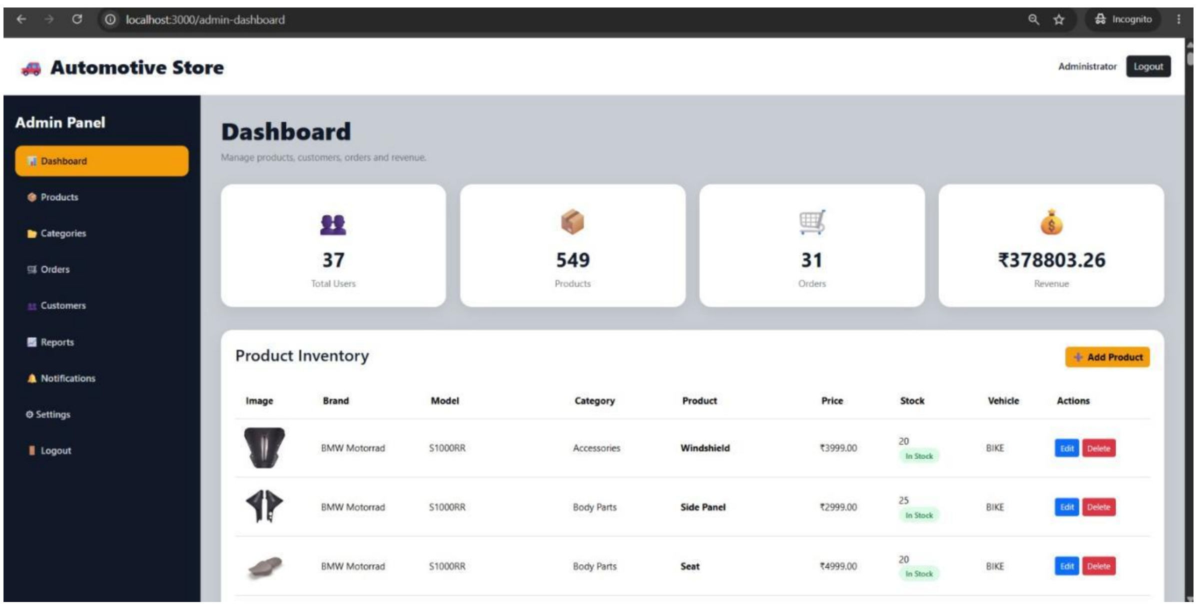
8. Application Screenshots — Admin Module

The administrator has complete control over the website through a dedicated admin panel.

8.1 Dashboard

Displays key store metrics: Total Users, Total Products, Total Orders, and Total Revenue, along with the full product inventory.

Admin Dashboard

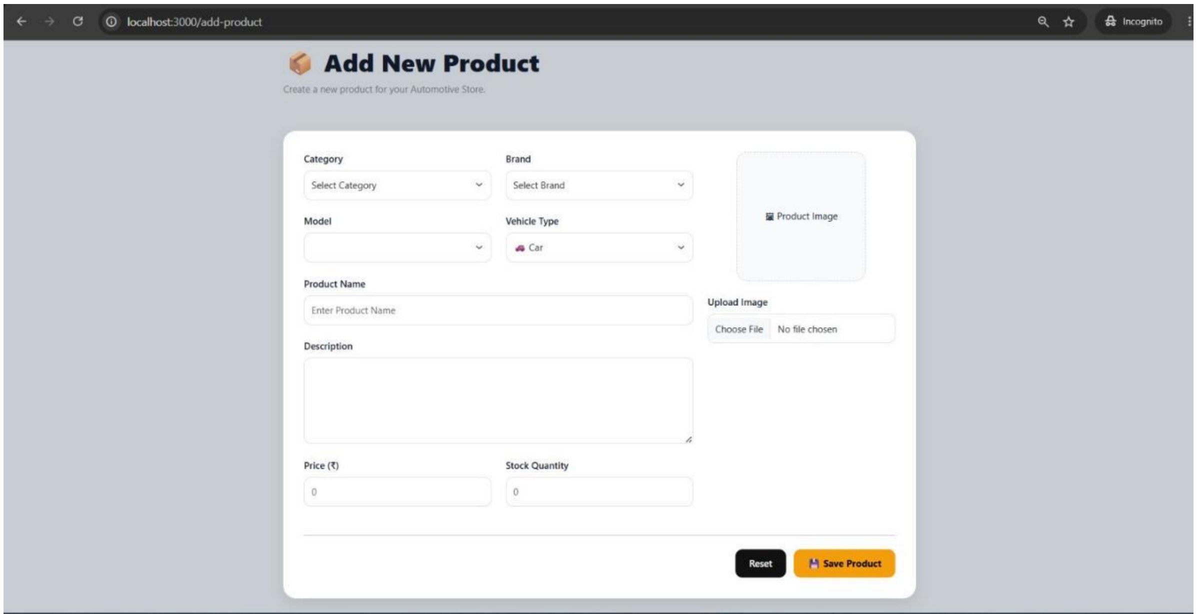


Screenshot: Admin dashboard with store metrics and inventory

8.2 Product Management

Administrators can add, edit, or delete products, including category, brand, model, vehicle type, price, stock, and product image.

Add New Product

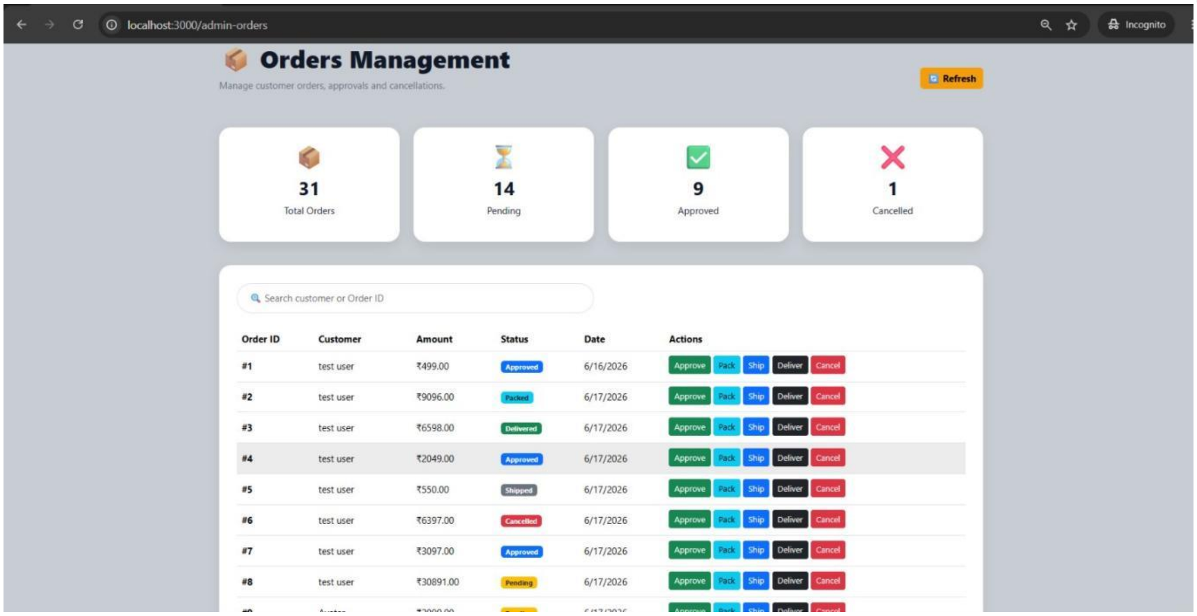


Screenshot: Add new product form

8.3 Order Management

Administrators can approve, pack, ship, deliver, or cancel customer orders from a centralized orders management screen.

Orders Management

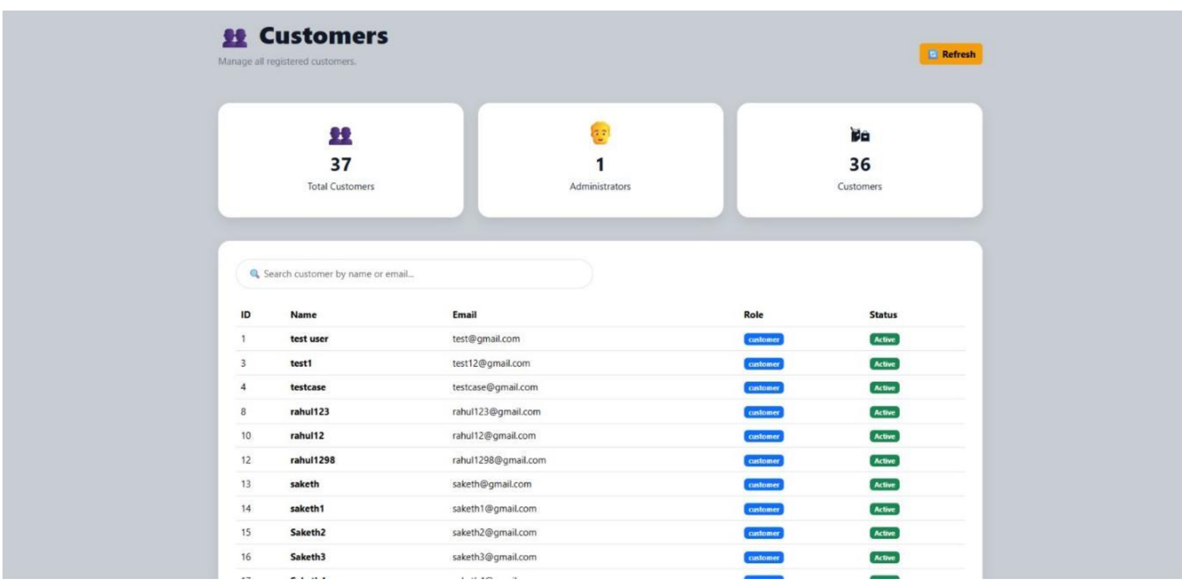


Screenshot: Admin order management with status actions

8.4 Customer Management

Administrators can view all registered customers, search by name or email, and see account roles and status.

Customers

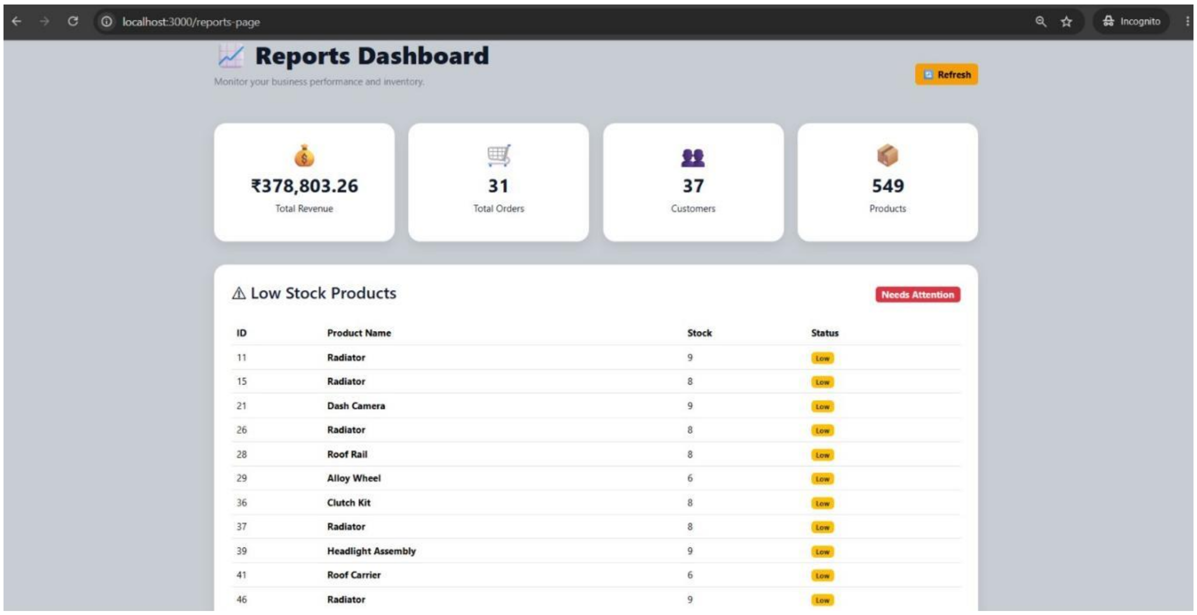


Screenshot: Registered customers list

8.5 Reports

The reports dashboard summarizes Total Revenue, Total Orders, Total Customers, Total Products, and highlights low-stock products needing attention.

Reports Dashboard

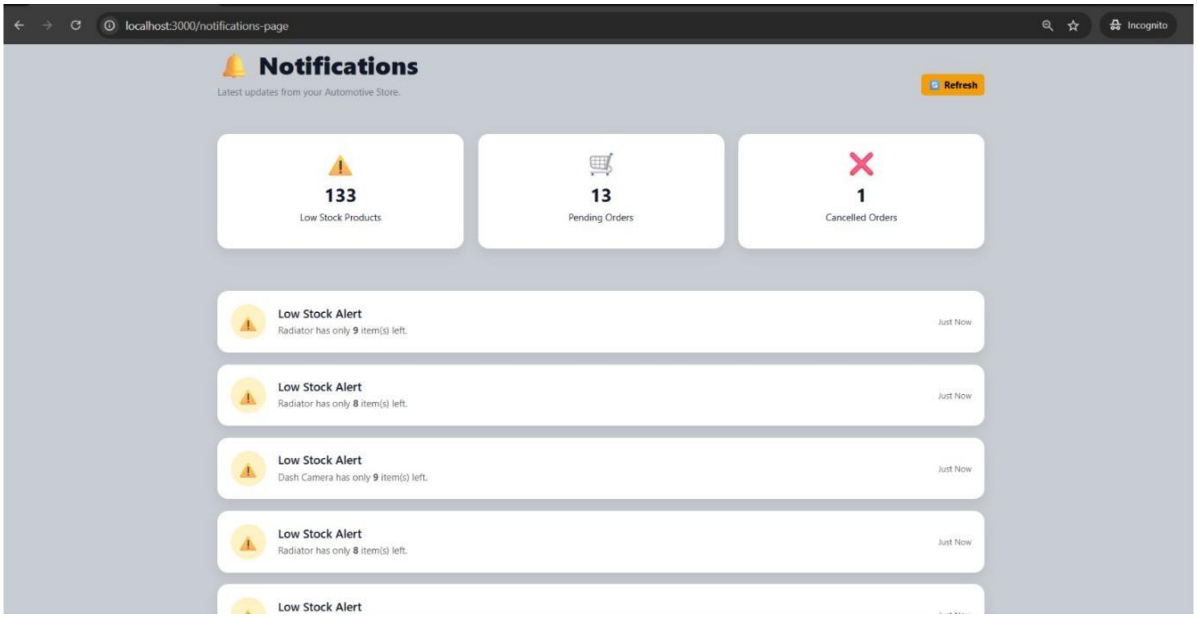


Screenshot: Reports dashboard with business performance metrics

8.6 Notifications

Displays real-time alerts for pending orders, cancelled orders, and low-stock products.

Notifications

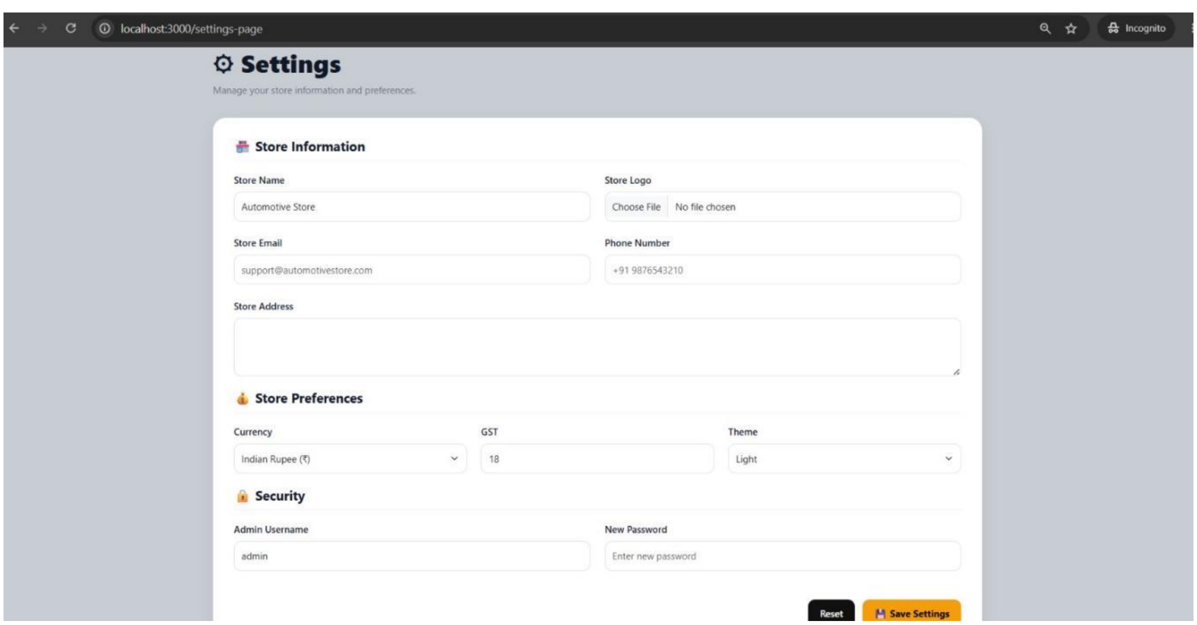


Screenshot: Store notifications and low-stock alerts

8.7 Store Settings

Administrators can update store information such as Store Name, Email, Phone, Address, currency, GST, theme, and admin credentials.

Settings



Screenshot: Store settings configuration

9. Application Workflow

The end-to-end flow of the application spans both the customer journey and the admin fulfillment process, as illustrated below.

9.1 Customer Order Workflow

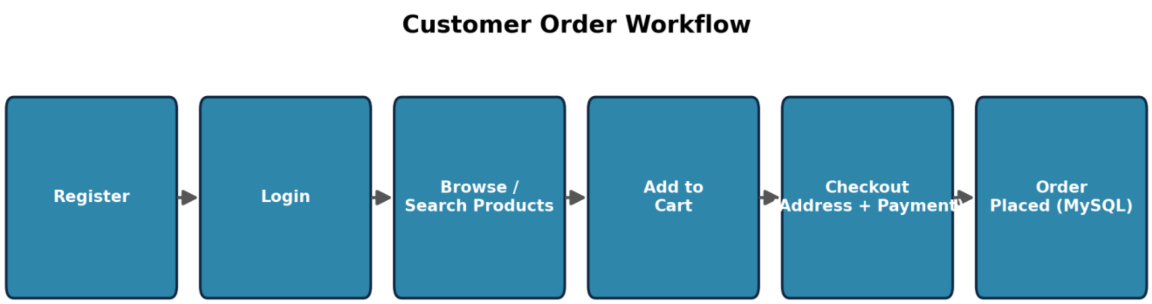


Figure 5: Customer journey from registration to order placement

9.2 Admin Order Fulfillment Workflow

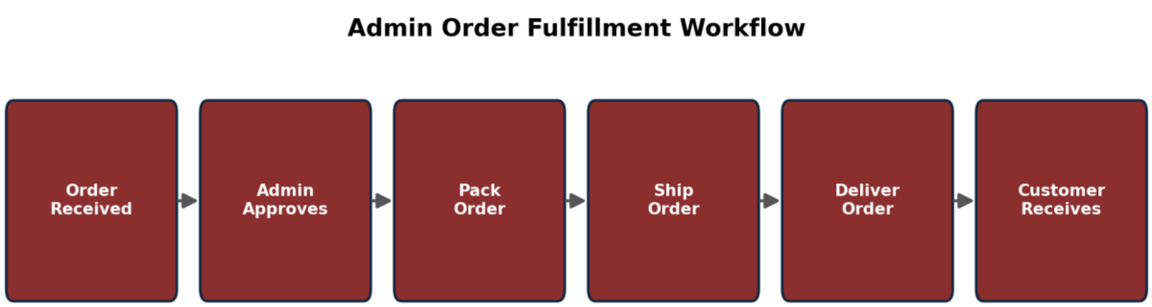


Figure 6: Admin order approval and fulfillment pipeline

10. Security Features

Password Hashing

Passwords are encrypted using bcrypt before being stored in the database.

Session Authentication

Only authenticated users can access protected pages.

Admin Authentication

Only administrators can access admin pages.

Helmet

Helmet secures HTTP headers to protect against common web vulnerabilities.

Rate Limiting

Limits repeated login requests to prevent brute-force attacks.

Environment Variables

Sensitive information such as session secrets and database credentials is stored inside the .env file.

11. Folder Structure & GitHub Repository

GitHub Repository: https://github.com/polagonitejagoud/Automotive-Parts-Store.git

The complete source code, commit history, and setup instructions (README) for the Automotive Parts Store are available at the repository link above.

```
Automotive-Parts-Store ├── app.js ├── package.json ├── .env ├── database/       db.js ├── middleware/       authMiddleware.js, adminMiddleware.js ├── routes/        authRoutes.js, productRoutes.js, cartRoutes.js, │               wishlistRoutes.js, orderRoutes.js, adminRoutes.js ├── uploads/ ├── public/        css/, js/, images/ └── views/         login.html, register.html, home.html, products.html, product-details.html, cart.html, wishlist.html,                profile.html, address.html, payment.html, tracking.html,                order-confirmed.html, admin-dashboard.html,                add-product.html, edit-product.html, customers.html,                categories.html, reports.html, notifications.html,                settings.html, admin-orders.html
```

12. Performance Test

12.1 Test Plan / Test Cases

The following modules were tested:

Test Case	Expected Result	Status
User Registration	User account created	Passed
User Login	Successful login	Passed
Admin Login	Admin dashboard opens	Passed
Product Search	Correct products displayed	Passed
Vehicle Filter	Filtered products shown	Passed
Add to Cart	Product added	Passed
Wishlist	Product saved	Passed
Checkout	Order created	Passed
Order Tracking	Status displayed	Passed
Profile Update	Profile updated	Passed
Product CRUD	Successful	Passed
Category CRUD	Successful	Passed
Reports	Generated correctly	Passed

12.2 Test Procedure

Testing was performed using:

- Manual Testing
- Browser Testing
- API Testing using Postman
- Database Validation using MySQL Workbench

Each module was tested individually and then integrated into the complete application.

12.3 Performance Outcome

The project performed successfully during testing.

- Registration works correctly.
- Login authentication is secure.
- Product search is responsive.
- Cart updates correctly.
- Orders are created successfully.
- Admin dashboard retrieves database information correctly.
- Reports display accurate statistics.
- Security middleware prevents unauthorized access.

The overall performance of the application is stable and suitable for deployment.

13. My Learnings

During this internship, I gained practical knowledge in:

- Full Stack Web Development
- Node.js
- Express.js
- MySQL Database Design
- HTML & CSS
- Bootstrap Framework
- JavaScript
- MVC Architecture
- REST API Development
- CRUD Operations
- Authentication & Authorization
- Password Hashing using bcrypt
- Session Management
- Helmet Security
- Rate Limiting
- Image Upload using Multer
- Git & GitHub
- Debugging
- Software Deployment
- Problem Solving

The internship greatly improved my understanding of industry-standard software development practices.

14. Future Work Scope

The current project can be enhanced with additional features such as:

- Online Payment Gateway Integration (Razorpay/Stripe)
- Email Notifications
- OTP Verification
- Invoice PDF Generation
- Product Reviews and Ratings
- AI-Based Product Recommendation
- Live Order Tracking
- Cloud Image Storage
- Mobile Application
- Multi-Vendor Marketplace
- Business Analytics Dashboard
- QR Code-Based Invoice
- Customer Support Chatbot
- Automated Backup and Recovery
- Continuous Integration and Continuous Deployment (CI/CD)

These enhancements will make the application more scalable, user-friendly, and suitable for commercial deployment.

15. Conclusion

The Automotive Parts Store project successfully demonstrates the development of a secure, full-stack e-commerce application using Node.js, Express.js, and MySQL, following the MVC architecture. It provides a complete online platform for purchasing automobile spare parts while enabling administrators to manage products, customers, and orders efficiently.

The project strengthened my understanding of web development, database management, authentication, security, and deployment practices. It also enhanced my problem-solving abilities and provided valuable experience in building industry-level software applications.

Project source code: https://github.com/polagonitejagoud/Automotive-Parts-Store.git